

Exercises 32.2-1 to 32.2-4

1 Exercise 32.2-1

Question 32.2-1

Working modulo $q = 11$, how many spurious hits does the Rabin–Karp matcher encounter in the text

$$T = 3141592653589793$$

when looking for the pattern

$$P = 26?$$

Solution

We use:

$$P = 26, \quad q = 11.$$

The pattern hash is

$$p = 26 \pmod{11} = 4.$$

So every text window of length 2 whose value is congruent to 4 (mod 11) will be a **hit**. A hit is:

- a **true hit** if the window is really 26,
- a **spurious hit** otherwise.

Now list all length-2 windows of the text:

$$31, 14, 41, 15, 59, 92, 26, 65, 53, 35, 58, 89, 97, 79, 93.$$

Compute each modulo 11:

$$\begin{aligned} 31 &\equiv 9, & 14 &\equiv 3, & 41 &\equiv 8, & 15 &\equiv 4, \\ 59 &\equiv 4, & 92 &\equiv 4, & 26 &\equiv 4, & 65 &\equiv 10, \\ 53 &\equiv 9, & 35 &\equiv 2, & 58 &\equiv 3, & 89 &\equiv 1, \\ 97 &\equiv 9, & 79 &\equiv 2, & 93 &\equiv 5. \end{aligned}$$

So the windows whose hash equals the pattern hash 4 are:

$$15, 59, 92, 26.$$

Among these:

- 26 is the real match,
- 15, 59, 92 are not equal to 26, so they are spurious hits.

Hence the number of spurious hits is

$$\boxed{3.}$$

Final Answer

The Rabin–Karp matcher encounters

3 spurious hits

at the windows

15, 59, 92.

2 Exercise 32.2-2

Question 32.2-2

How would you extend the Rabin–Karp method to the problem of searching a text string for an occurrence of any one of a given set of k patterns? Start by assuming that all k patterns have the same length. Then generalize your solution to allow the patterns to have different lengths.

Solution

We solve the problem in two stages.

Case 1: All k patterns have the same length m

Suppose the patterns are:

$$P_1, P_2, \dots, P_k,$$

and each has length m .

Idea

Instead of comparing the text window hash with one pattern hash, we compare it with the set of all pattern hashes.

Method

1. Compute the hash of each pattern:

$$h_i = \text{hash}(P_i) \pmod{q}, \quad i = 1, 2, \dots, k.$$

2. Store these hashes in a hash table or dictionary.
3. Run Rabin–Karp on the text using windows of length m .
4. For each window hash t_s :
 - if t_s is not in the set of pattern hashes, reject immediately;
 - if t_s matches one or more stored hashes, verify the actual substring against the corresponding patterns.

Why it works

A real occurrence of some pattern must have the same hash as that pattern. So every real match is detected as a hit. Then direct verification removes spurious hits.

Time Complexity

- Preprocessing pattern hashes:

$$\mathcal{O}(km).$$

- Text scan with rolling hash:

$$\mathcal{O}(n - m + 1) = \mathcal{O}(n).$$

- Verification happens only when a hit occurs.

So the total expected running time is roughly

$$\boxed{\mathcal{O}(km + n)}$$

plus verification cost.

Case 2: Patterns have different lengths

Now suppose the pattern lengths are not all equal.

Let the distinct lengths be:

$$m_1, m_2, \dots, m_r.$$

Method

Group the patterns by length.

For each length m_j :

1. collect all patterns of that length,
2. compute their hashes,
3. run Rabin–Karp on the text using windows of length m_j .

So we perform one Rabin–Karp scan for each distinct length.

Why this works

Rabin–Karp requires a fixed window length. If patterns have different lengths, then each distinct length needs its own rolling-hash scan.

Time Complexity

If there are r distinct lengths, then:

$$\text{total scan time} = \mathcal{O}(rn),$$

plus preprocessing of all patterns:

$$\mathcal{O}\left(\sum_{i=1}^k |P_i|\right).$$

So a clean bound is:

$$\boxed{\mathcal{O}\left(rn + \sum_{i=1}^k |P_i|\right)}$$

plus verification cost.

Final Answer

Same length case: hash all k patterns, store the hashes, and compare each text window hash against this set.

Different lengths case: group patterns by length and run a separate Rabin–Karp scan for each distinct length.

This correctly finds occurrences of any one of the k patterns.

3 Exercise 32.2-3

Question 32.2-3

Show how to extend the Rabin–Karp method to handle the problem of looking for a given $m \times m$ pattern in an $n \times n$ array of characters. The pattern may be shifted vertically and horizontally, but it may not be rotated.

Solution

We want to search for a 2D pattern inside a 2D text.

Let:

$$P = \text{an } m \times m \text{ pattern,} \quad T = \text{an } n \times n \text{ text array.}$$

Main Idea

The ordinary Rabin–Karp algorithm hashes a 1D string. For 2D matching, we compute a **2D rolling hash**.

The basic idea is:

- first hash each row,
- then combine the row hashes into a single block hash.

So the $m \times m$ block gets a single hash value.

Step 1: Row Hashes

Treat each row as a radix- d string.

For each row of the pattern, compute its hash:

$$r_i = \text{hash of row } i, \quad i = 1, 2, \dots, m.$$

Similarly, for each row of the text, compute rolling hashes of every length- m horizontal window. So for each text row, we can compute the hash of every horizontal segment of length m in $\mathcal{O}(1)$ update time.

Step 2: Combine Rows into a Block Hash

Now each $m \times m$ block consists of m row-hashes stacked vertically.

Treat these row-hashes as digits in another radix system and combine them:

$$H = \text{hash}(r_1, r_2, \dots, r_m).$$

This gives one hash for the whole $m \times m$ block.

Step 3: Rolling in Two Directions

We use rolling updates in two ways:

- **Horizontal rolling:** update row-window hashes as we move right.
- **Vertical rolling:** update block hashes as we move down.

So once the first block hash is computed, every neighboring block can be updated efficiently.

Algorithm Outline

1. Compute the hash of each row of the pattern.
2. Combine these into one pattern-block hash.
3. For each text row, compute hashes of all horizontal windows of length m .
4. For each column position, combine m consecutive row hashes into a vertical block hash.
5. Compare each text-block hash with the pattern-block hash.
6. If the hash matches, verify the actual $m \times m$ block.

Why it works

A true occurrence of the pattern must produce the same 2D hash as the pattern. If the hash matches, direct checking confirms whether it is a real occurrence or a spurious hit.

Time Complexity

There are roughly

$$(n - m + 1)^2$$

possible positions for the $m \times m$ pattern.

Using rolling hashes, each block hash can be updated in constant time after preprocessing. Thus the total scan time is

$$\boxed{\mathcal{O}(n^2)}$$

plus verification cost.

Final Answer

Extend Rabin–Karp to two dimensions by:

- hashing rows first,
- then hashing sequences of row-hashes vertically.

This gives a rolling 2D hash for each $m \times m$ block. Compare block hashes with the pattern hash, and verify only on a hit.

4 Exercise 32.2-4

Question 32.2-4

Alice has a copy of a long n -bit file

$$A = \langle a_{n-1}, a_{n-2}, \dots, a_0 \rangle,$$

and Bob similarly has

$$B = \langle b_{n-1}, b_{n-2}, \dots, b_0 \rangle.$$

Alice and Bob wish to know if their files are identical. To avoid transmitting all of A or B , they choose a prime

$$q > 1000n$$

and randomly choose an integer

$$x \in \{0, 1, \dots, q-1\}.$$

Then Alice evaluates

$$A(x) = \left(\sum_{i=0}^{n-1} a_i x^i \right) \pmod{q}$$

and Bob similarly evaluates

$$B(x) = \left(\sum_{i=0}^{n-1} b_i x^i \right) \pmod{q}.$$

Prove that if $A \neq B$, there is at most one chance in 1000 that

$$A(x) = B(x),$$

whereas if the two files are the same, $A(x)$ is necessarily the same as $B(x)$.

Solution

We consider two cases.

Case 1: $A = B$

If the files are identical, then for every i ,

$$a_i = b_i.$$

So the two polynomials are exactly the same:

$$A(x) = B(x)$$

for every choice of x .

Hence if $A = B$, then necessarily

$$\boxed{A(x) = B(x)}.$$

Case 2: $A \neq B$

If the files are different, then there exists at least one index i such that

$$a_i \neq b_i.$$

Define the difference polynomial:

$$D(x) = A(x) - B(x) \pmod{q}.$$

Then $D(x)$ is **not** the zero polynomial, because $A \neq B$.

Also, the degree of $D(x)$ is at most

$$n - 1.$$

Key Fact

A nonzero polynomial of degree at most $n - 1$ over a field can have at most

$$n - 1$$

roots modulo q .

So the equation

$$A(x) = B(x)$$

is equivalent to

$$D(x) = 0,$$

and this can happen for at most $n - 1$ values of x .

Probability Bound

The value x is chosen uniformly from

$$\{0, 1, \dots, q - 1\},$$

which has q possible values.

Therefore,

$$\mathbb{P}(A(x) = B(x)) \leq \frac{n - 1}{q}.$$

Since

$$q > 1000n,$$

we have

$$\frac{n - 1}{q} < \frac{n}{1000n} = \frac{1}{1000}.$$

So:

$$\mathbb{P}(A(x) = B(x) \mid A \neq B) < \frac{1}{1000}.$$

Hence there is at most one chance in 1000 that two different files produce the same value.

Final Conclusion

- If $A = B$, then

$$A(x) = B(x) \text{ for every } x.$$

- If $A \neq B$, then

$$\mathbb{P}(A(x) = B(x)) < \frac{1}{1000}.$$

So this is a correct randomized fingerprint test for equality of two files.